UNIVERSITY OF KELANIYA / SENG 34213

FOREVER HOTEL MANAGEMENT SYSTEM

Performance Test Results

Six-Subsystem Functional and Load-Test Evidence

Reporting date: 25 October 2026. Evidence is identified by subsystem branch, commit, and run date; comparisons across different revisions are not a single hotel-wide release test.

Contents

1. Performance measurement status

2. Hotel Website (HW)

3. Manager Dashboard (MAD)

4. Front Desk System (FDS)

5. Food Ordering and Service Requests (FOSS)

6. Kitchen Management System (KMS)

7. Worker Management System (WKMS)

8. Integrated performance test protocol

---

1. Performance measurement status

None of the six evidence sets includes a complete approved hotel-wide performance run with documented response-time percentiles, throughput, error rates and an integrated release configuration. The correct load-test acceptance status is therefore NOT TESTED / NOT VERIFIED. Passing CI functional tests and the presence of a performance harness are recorded separately.

Subsystem	Dedicated executed benchmark	Recorded p95 / throughput / errors	Status
HW	No completed benchmark evidenced	Not available	NOT VERIFIED
MAD	Harness present; execution unverified	Not available	NOT VERIFIED
FDS	No completed benchmark evidenced	Not available	NOT VERIFIED
FOSS	No completed benchmark evidenced	Not available	NOT VERIFIED
KMS	No completed benchmark evidenced	Not available	NOT VERIFIED
WKMS	No completed benchmark evidenced	Not available	NOT VERIFIED

Relevant performance acceptance targets

Original SRS NFR-01 sets a target of at least 95% of API requests within 500 ms under 100 concurrent users; this is a required target, not an achieved result. Realtime event targets such as inter-service propagation within one second require synchronized producer, broker, consumer and client timestamps. Confirm exact applicability per endpoint with the approved SRS before signing off.

Evidence distinction

A standalone health endpoint check establishes response correctness at the time of the test, not the distribution of response times or supported concurrency. A repository-supplied load test harness establishes an available test method, not that the harness was executed.

2. Hotel Website (HW)

Repository: forever-hotel/hotel-website | Branch: pre-develop | Tested revision: 606acf0674a18621639bce572ce3577af933cd63 | CI date: 22 October 2026

Evidence: https://github.com/forever-hotel/hotel-website/actions/runs/39726186380, https://github.com/forever-hotel/hotel-website/actions/runs/39726186472, https://github.com/forever-hotel/hotel-website/actions/runs/39726186469

The container CI checks correct readiness behavior during a PostgreSQL outage, not recovery latency. Booking/payment actions cannot be load-accepted on the evaluated pre-develop code.

Documented performance scenarios

Scenario	Operation / path	Required measurements / objective	Current status
HW-P01	Public room catalogue	p50/p95 HTTP latency, throughput, HTTP errors and response integrity	Not tested
HW-P02	Room-type availability	Calendar/capacity reads, p95 and concurrent inventory consistency	Not tested
HW-P03	Guest registration and login	Authentication latency, failure rate and sensitive-data handling	Not tested
HW-P04	Website first render	Page-load and visual timing on public routes	Not tested
HW-P05	Online booking and payment	End-to-end payment/booking flow - blocked on absent complete pre-develop implementation	Not tested
HW-P06	Container/DB outage recovery	Readiness 503 correctness covered by container CI; latency and recovery time unmeasured	Not tested

Measured outcomes

Required result	Current evidence
Test environment, datasets and run ID	Not recorded
Concurrency profile and duration	Not measured
p50 / p95 / p99 (milliseconds)	Not measured
Throughput (requests per second)	Not measured
Failed requests / timeout rate	Not measured
Broker / Socket.IO propagation delay	Not measured
SRS pass/fail determination	Not evaluated

The scenario list documents required tests and does not assert execution. Production/shared hotel data must not be used for unapproved load testing.

3. Manager Dashboard (MAD)

Repository: forever-hotel/manager-dashboard | Branch: development | Tested revision: bba65a0762286d4b0ac902865d7a2dd1e683e9c7 | CI date: 24 October 2026

Evidence: https://github.com/forever-hotel/manager-dashboard/actions/runs/40137091073, https://github.com/forever-hotel/manager-dashboard/actions/runs/39847903811

The repository has backend/test/performance/run.ts, metrics.ts and evaluate.ts. It defines 100- and 300-user read-only profiles with a 5-minute warm-up and 15-minute measurement window, per-route percentiles and independently correlated event/UI timing. Raw, approved benchmark results were not retrieved; unit tests of metrics calculations are not benchmark results.

Documented performance scenarios

Scenario	Operation / path	Required measurements / objective	Current status
MAD-P01	100 authenticated-user baseline	13 read-only API routes and socket sessions; warm-up 5min then measure 15min	Not tested
MAD-P02	300 authenticated-user capacity comparison	Compare 300-user p95 with 100-user p95, errors, disconnects and scheduling completeness	Not tested
MAD-P03	Realtime event propagation	Measure commit-to-client event arrival under synchronized clocks	Not tested
MAD-P04	Browser visibility	Correlate change receipt to actual rendered UI visibility	Not tested
MAD-P05	Staging evaluator	Execute checked-in metrics/evaluate tooling against accepted raw outputs	Not tested

Manager Dashboard performance harness specification

Profile	Repository-defined plan
Baseline	100 authenticated manager sessions with distinct tokens
Comparison	300 authenticated manager sessions
Warm-up	300,000 ms; excluded from recorded samples
Measurement	900,000 ms
Request mix	13 read-only management endpoints and Socket.IO
Recorded metrics (planned)	p50, p95, p99, error rate, disconnects, >=500ms fraction, per-route latency
100-user target	p95 <=500ms, complete scheduled requests and no errors/disconnects
300-user comparator	p95 <=1.1 x successful 100-user baseline
Event / UI targets	Correlate delivery and browser render timestamps against 1s/2s targets
Run output	performance-results/<run-label>/summary.json and receipts.json

Measured outcomes

Required result	Current evidence
Test environment, datasets and run ID	Not recorded
Concurrency profile and duration	Not measured
p50 / p95 / p99 (milliseconds)	Not measured
Throughput (requests per second)	Not measured
Failed requests / timeout rate	Not measured
Broker / Socket.IO propagation delay	Not measured
SRS pass/fail determination	Not evaluated

The scenario list documents required tests and does not assert execution. Production/shared hotel data must not be used for unapproved load testing.

4. Front Desk System (FDS)

Repository: forever-hotel/front-desk | Branch: develop | Tested revision: 503465b7eb2c29b56cc2ec0d329d91d6b40853d4 | CI date: 24 October 2026

Evidence: https://github.com/forever-hotel/front-desk/actions/runs/40057114169, https://github.com/forever-hotel/front-desk/actions/runs/40057114175

No executed k6 or equivalent measurement was evidenced. CI validates logic and transactions, not 100-user latency, Socket.IO fan-out delay or real RabbitMQ-FDS-WKMS propagation timing.

Documented performance scenarios

Scenario	Operation / path	Required measurements / objective	Current status
PERF-FDS-01	Booking search / arrivals / departures	HTTP p50/p95, throughput and error rate	Not tested
PERF-FDS-02	Room status board reads	HTTP p50/p95 and throughput	Not tested
PERF-FDS-03	Authorized JWT API reads	Authentication overhead and 401/403 correctness	Not tested
PERF-FDS-04	Check-in with synthetic booking and room	Transaction latency and PostgreSQL lock contention	Not tested
PERF-FDS-05	Room change on synthetic assignments	Transaction latency and conflicting reassignment detection	Not tested
PERF-FDS-06	Checkout with synthetic folio and payments	Commit latency, post-commit event and side-effect behavior	Not tested
PERF-FDS-07	Socket.IO room-status delivery	Event publish-to-client p50/p95 latency	Not tested
PERF-FDS-08	RabbitMQ checkout and WKMS escalation	Producer/consumer delay, failure and retry behavior	Not tested

Measured outcomes

Required result	Current evidence
Test environment, datasets and run ID	Not recorded
Concurrency profile and duration	Not measured
p50 / p95 / p99 (milliseconds)	Not measured
Throughput (requests per second)	Not measured
Failed requests / timeout rate	Not measured
Broker / Socket.IO propagation delay	Not measured
SRS pass/fail determination	Not evaluated

The scenario list documents required tests and does not assert execution. Production/shared hotel data must not be used for unapproved load testing.

5. Food Ordering and Service Requests (FOSS)

Repository: forever-hotel/food-ordering | Branch: develop / feature/34-booking-room-verification | Tested revision: 17af08411e / 343b7d0018 | CI date: 13 October 2026

Evidence: https://github.com/forever-hotel/food-ordering/actions/runs/38381405941, https://github.com/forever-hotel/food-ordering/actions/runs/38419260802

Functional NestJS and guest session tests exist, including mock booking verification on a separate feature revision, but no real KMS guest ordering load workflow or accepted session stress data was available.

Documented performance scenarios

Scenario	Operation / path	Required measurements / objective	Current status
FOSS-P01	Guest authentication and booking verification	Valid/invalid/revoked sessions and owner-service response-time	Not tested
FOSS-P02	Menu browsing and cart operations	Guest UI responsiveness, data consistency and cache behavior	Not tested
FOSS-P03	Food-order bursts and duplicates	Order write throughput and KMS acknowledgement/idempotency	Not tested
FOSS-P04	Service requests and worker dispatch	FOSS-WKMS handoff and state propagation	Not tested
FOSS-P05	Check-in/checkout session lifetime	Session activation, revocation and rejected stale session timing	Not tested
FOSS-P06	Failure and recovery	Outage-induced timeouts, broker backlog and recovery	Not tested

Measured outcomes

Required result	Current evidence
Test environment, datasets and run ID	Not recorded
Concurrency profile and duration	Not measured
p50 / p95 / p99 (milliseconds)	Not measured
Throughput (requests per second)	Not measured
Failed requests / timeout rate	Not measured
Broker / Socket.IO propagation delay	Not measured
SRS pass/fail determination	Not evaluated

The scenario list documents required tests and does not assert execution. Production/shared hotel data must not be used for unapproved load testing.

6. Kitchen Management System (KMS)

Repository: forever-hotel/kitchen-management | Branch: develop | Tested revision: 89087002ac2df8f07b6140c00111979c75bc2799 | CI date: 10 October 2026

Evidence: https://github.com/forever-hotel/kitchen-management/actions/runs/38038213541

Basic scaffold, persistence mappings and PostgreSQL health E2E checks pass. No verified order processing, KOT printer load, or FOSS-KMS-WKMS propagation benchmark was present at the evaluated develop snapshot.

Documented performance scenarios

Scenario	Operation / path	Required measurements / objective	Current status
KMS-P01	Menu read	Category/menu list p50/p95/p99, error rates and database query time	Not tested
KMS-P02	Incoming order burst	Order correctness, duplicates and database performance	Not tested
KMS-P03	Order status updates	Concurrent kitchen actions, atomicity and events	Not tested
KMS-P04	KOT delivery	Print queue length, retries, output and duplicate prevention	Not tested
KMS-P05	Guest to kitchen to worker	FOSS-KMS-WKMS timestamped cross-service propagation	Not tested
KMS-P06	Failure and recovery	DB, broker and printer outage recovery, missing events	Not tested

Measured outcomes

Required result	Current evidence
Test environment, datasets and run ID	Not recorded
Concurrency profile and duration	Not measured
p50 / p95 / p99 (milliseconds)	Not measured
Throughput (requests per second)	Not measured
Failed requests / timeout rate	Not measured
Broker / Socket.IO propagation delay	Not measured
SRS pass/fail determination	Not evaluated

The scenario list documents required tests and does not assert execution. Production/shared hotel data must not be used for unapproved load testing.

7. Worker Management System (WKMS)

Repository: forever-hotel/worker-management | Branch: develop | Tested revision: 0f91f1c9d6390c80c9f12f82b834fee9bf374104 | CI date: 25 October 2026

Evidence: https://github.com/forever-hotel/worker-management/actions/runs/40174879698

Functional task and Socket.IO tests do not establish sustained load, task-concurrency latency or deployed event delivery timing. The tested backend includes skipped suites and is not a complete integration benchmark.

Documented performance scenarios

Scenario	Operation / path	Required measurements / objective	Current status
WKMS-P01	Worker task queue	GET /wkms/tasks/queue; latency and error rate	Not tested
WKMS-P02	Task claim concurrency	POST /wkms/tasks/:id/claim; max-three rule, double-claim prevention	Not tested
WKMS-P03	Worker start / complete	Transactional throughput, lock waits, errors	Not tested
WKMS-P04	RabbitMQ checkout event	FDS checkout to WKMS cleaning task propagation delay	Not tested
WKMS-P05	Socket.IO task broadcast	task.created/claimed/escalated fan-out and UI update time	Not tested
WKMS-P06	Shift dashboard	GET /wkms/shift/summary; response-time distribution	Not tested

Measured outcomes

Required result	Current evidence
Test environment, datasets and run ID	Not recorded
Concurrency profile and duration	Not measured
p50 / p95 / p99 (milliseconds)	Not measured
Throughput (requests per second)	Not measured
Failed requests / timeout rate	Not measured
Broker / Socket.IO propagation delay	Not measured
SRS pass/fail determination	Not evaluated

The scenario list documents required tests and does not assert execution. Production/shared hotel data must not be used for unapproved load testing.

8. Integrated performance test protocol

1. Select an approved staging release consisting of mutually compatible versions of all six backend services, frontend applications, gateway, database schema and broker contracts.

2. Provision a disposable database/broker stack, synthetic guest/worker/staff identities, safe test inventory, payment stubs and isolated email/printing endpoints.

3. Record deployment commit SHAs, environment CPU/RAM, Node/PostgreSQL/broker versions, network region, fixture size and request mix before execution.

4. Run graduated concurrency profiles (1, 10, 25, 50 and 100 virtual users); stop on predefined critical failures and never mutate shared development or production hotel records.

5. Export raw scripts, JSON/CSV metrics and timestamped HTTP, database, broker and Socket.IO traces. For write operations, use unique synthetic bookings/order/task identities.

6. Compute p50/p95/p99, requests per second, failed requests, transaction conflicts, broker lag and recovery time. Compare the measurements with agreed NFR acceptance values.

7. Document failures and reviewer decisions. Replace NOT TESTED status only with matching raw evidence and signed acceptance.

No pass/fail performance verdict can be made without the missing benchmark measurements.